CS336 Assignment 2: Systems and Parallelism

Writeup

Your Name
SUNet ID

Spring 2026

Contents

1 Profiling and Benchmarking

1.1 End-to-End Benchmarking

Problem (benchmarking_script)

(b) Timings for forward, backward, and optimizer step. Table 1 summarizes the mean timings across four model sizes (5 warm-up steps, 10 measurement iterations each, batch size 4, context length 512). The 10B model (12.8B params) OOM'd on a single 96 GB GPU even at batch size 1.

As model size grows from small (129M) to xl (3.4B), forward time scales from 8.0ms to 158ms (∼20×), backward from 24ms to 605ms (∼25×), and optimizer from 28ms to 364ms (∼13×). The backward-to-forward ratio ranges from ∼3× (small) to ∼3.8× (xl), higher than the theoretical 2× due to the memory-bound nature of non-matmul backward kernels at these model scales.

Table 1: Mean benchmark timings across model sizes (5 warmup, 10 measurement steps, seconds).

Size	Params	Forward	Backward	Optimizer	Total
small	128.6M	0.0080	0.0240	0.0278	0.0645
medium	423.2M	0.0163	0.1005	0.0491	0.1727
large	969.4M	0.0606	0.2242	0.0883	0.3824
xl	3406.8M	0.1577	0.6049	0.3645	1.1884
10B	12832.8M	OOM (single GPU, 96 GB)			

Detailed per-iteration timings for each model size are reported in Tables 2–5.

Table 2: Per-iteration timings — small (128.6M params, seconds).

Iter	Forward	Backward	Optimizer	Total
0	0.008805	0.027929	0.027512	0.068860
1	0.007973	0.023553	0.027840	0.063957
2	0.007969	0.023524	0.027921	0.064068
3	0.007997	0.023520	0.027875	0.064041
4	0.007806	0.023695	0.027844	0.063934
5	0.007815	0.023712	0.027913	0.064079
6	0.007880	0.023598	0.027878	0.063962
7	0.007954	0.023538	0.027915	0.064049
8	0.007936	0.023600	0.027847	0.064024
9	0.008003	0.023536	0.027891	0.064026

Table 3: Per-iteration timings — medium (423.2M params, seconds).

Iter	Forward	Backward	Optimizer	Total
0	0.022665	0.103150	0.049132	0.181706
1	0.015675	0.100775	0.049791	0.172990
2	0.015603	0.101244	0.049703	0.173358
3	0.015699	0.100482	0.050023	0.173115
4	0.015801	0.100800	0.049977	0.173389
5	0.015657	0.100513	0.048556	0.171523
6	0.015449	0.098595	0.048483	0.169038
7	0.015732	0.100194	0.048560	0.171374
8	0.015141	0.098973	0.048456	0.168943
9	0.015582	0.100286	0.048559	0.171229

Table 4: Per-iteration timings — large (969.4M params, seconds).

Iter	Forward	Backward	Optimizer	Total
0	0.069621	0.225146	0.087279	0.391637
1	0.059705	0.224016	0.088500	0.381821
2	0.059492	0.224260	0.088216	0.381226
3	0.059803	0.223808	0.088319	0.381162
4	0.059690	0.224178	0.088502	0.381656
5	0.059637	0.223993	0.088518	0.381416
6	0.059546	0.223956	0.088468	0.381155
7	0.059490	0.224351	0.088252	0.381292
8	0.059381	0.224349	0.088170	0.381223
9	0.059410	0.224161	0.088399	0.381158

Table 5: Per-iteration timings — xl (3.41B params, seconds).

Iter	Forward	Backward	Optimizer	Total
0	0.213040	0.609433	0.361567	1.244934
1	0.151633	0.603544	0.364648	1.181239
2	0.151771	0.604308	0.365424	1.182898
3	0.151549	0.604199	0.365209	1.182638
4	0.151668	0.604337	0.364411	1.181654
5	0.151368	0.605109	0.365054	1.182857
6	0.151595	0.603789	0.364284	1.181087
7	0.151271	0.605567	0.364650	1.182793
8	0.151552	0.604438	0.364929	1.182570
9	0.151519	0.604381	0.364340	1.181539

(c) Effect of removing warm-up steps. We re-run the large model (969M params) with warmup $\in \{0, 1, 5\}$ to isolate the effect of warm-up on measured timings (Table 6).

Without warm-up (warmup=0), the first iteration is dramatically slower: the forward pass takes 295ms compared to ∼62ms in steady state (a ∼4.8× slowdown), and the backward pass reaches 286ms vs. ∼224ms. The total for iteration 0 is 629ms—1.6× the steady-state ∼384ms. This is because CUDA performs lazy initialization on the first kernel launch, including context creation, kernel JIT compilation, and memory pool allocation.

With warmup=1, the first measured iteration still shows overhead (73ms forward vs. ∼60ms steady state), because a single warm-up step only partially amortizes cuDNN autotuning and memory allocator warm-up. By warmup=5 (our default), the overhead on iteration 0 shrinks to 69ms forward—only ∼16% above steady state—and iterations 1–9 are fully stable at ∼381ms total.

The steady-state timings (iterations 4–9) are consistent across all warm-up settings at ∼381–385ms total, confirming that warm-up only affects early measurements and 5 warm-up steps are sufficient for reliable benchmarking.

Table 6: Warm-up effect on the large model (969M params, seconds).

(a) warmup = 0

Iter	Forward	Backward	Optimizer	Total
0	0.294910	0.285918	0.039139	0.629414
1	0.063381	0.236767	0.091641	0.401459
2	0.063882	0.228353	0.090433	0.392371
3	0.063467	0.226980	0.090378	0.390437
4	0.062608	0.224941	0.088858	0.386012
5	0.062538	0.224086	0.088961	0.385279
6	0.061851	0.223798	0.088168	0.383509
7	0.061446	0.224394	0.088809	0.384442
8	0.061604	0.223993	0.088839	0.384135
9	0.062133	0.224124	0.089207	0.385142

(b) warmup = 1

Iter	Forward	Backward	Optimizer	Total
0	0.072542	0.225798	0.087844	0.395896
1	0.061591	0.223565	0.088606	0.383297
2	0.061285	0.223906	0.088446	0.383004
3	0.062142	0.223773	0.088503	0.383811
4	0.059694	0.224057	0.088454	0.381622
5	0.059697	0.224042	0.088680	0.382038
6	0.059696	0.223751	0.088694	0.381424
7	0.059165	0.224367	0.088501	0.381191
8	0.059721	0.223903	0.088787	0.381676
9	0.059531	0.224207	0.088459	0.381434

(c) warmup = 5 (default)

Iter	Forward	Backward	Optimizer	Total
0	0.069072	0.225348	0.087045	0.390757
1	0.059510	0.224080	0.088312	0.381124
2	0.059796	0.223846	0.088253	0.381137
3	0.060320	0.223399	0.088299	0.381211
4	0.059558	0.224158	0.088539	0.381545
5	0.059449	0.224155	0.088413	0.381281
6	0.059837	0.224020	0.088368	0.381433
7	0.059941	0.223970	0.088252	0.381344
8	0.059600	0.224101	0.088168	0.381192
9	0.059407	0.224022	0.088702	0.381272

1.2 Nsight Systems Profiling

Problem (nsys_profile)

We profiled two model sizes—**small** (128.6M params) and **large** (969.4M params)—at three power-of-two context lengths each, using `nsys profile` with 5 warmup + 5 measurement iterations. The small model was profiled at context lengths 256, 1024, and 2048 (the maximum that fits in GPU memory); the large model at 256, 512, and 1024.

(a) Total time on forward pass. Table 7 shows the GPU-projected forward pass time per iteration from the `nvtx_gpu_proj_sum` report. The GPU-projected time measures the actual GPU execution span of kernels launched within the `forward_pass` NVTX range, and is generally *higher* than the Python `timeit` measurement (which only records CPU dispatch time without `torch.cuda.synchronize()`). For example, for the large model at context length 512, nsys reports a GPU forward time of 111.2 ms, while `timeit` records ∼60.6 ms—reflecting only how long the CPU takes to dispatch forward kernels asynchronously. The wall-clock total per iteration (∼382 ms with synchronization) is consistent with the sum of GPU forward, backward, and optimizer times.

Table 7: GPU-projected forward pass time per iteration (ms).

Model	Context	GPU Forward (ms)	Python `timeit` Forward (ms)
small	256	11.9	—
small	1024	50.9	—
small	2048	145.3	—
large	256	60.0	—
large	512	111.2	60.6 (ctx=512 benchmark)
large	1024	300.4	—

The forward pass time grows roughly quadratically with context length (e.g., small: 11.9 → 50.9 → 145.3 ms for 256 → 1024 → 2048), consistent with the $O(T^2)$ attention scaling dominating at longer sequences.

(b) Most time-consuming CUDA kernel. The most time-consuming CUDA kernel across all profiles is the CUTLASS SGEMM kernel (single-precision general matrix multiply), which implements all matmul operations in the Transformer (linear projections, attention score computation, FFN layers). Table 8 shows the top kernel for each configuration.

Table 8: Top CUDA kernel by cumulative GPU time (full training step).

Model	Context	Top Kernel	Time %	Invocations
small	256	`sgemm_128x64_nn`	16.9%	840
small	1024	`sgemm_128x64_nn`	13.3%	840
small	2048	`sgemm_128x64_nn`	10.2%	840
large	256	`sgemm_128x256_tn`	24.3%	2530
large	512	`sgemm_128x256_tn`	23.3%	3240
large	1024	`sgemm_128x256_tn`	16.9%	2530

The same GEMM kernel class dominates both the forward-only and forward+backward profiles. During a full training step, GEMM kernels remain the most time-consuming category, though their collective share decreases relative to forward-only because the backward pass and optimizer introduce additional elementwise and reduction operations.

(c) Non-matmul kernels with non-trivial runtime. Table 9 shows that GEMM kernels account for 49–61% of total GPU kernel time during a full training step. The remaining time is spent on several non-matmul kernel categories:

- `vectorized_elementwise_kernel` (mul, add): 6–9% each, implementing LayerNorm scaling, residual additions, and activation functions.
- `elementwise_kernel` (div, subtract): 3–7%, used in softmax normalization and gradient computation.
- `reduce_kernel`: ∼3%, performing reductions for LayerNorm mean/variance and loss computation.

Table 9: GEMM kernel fraction of total GPU kernel time (full training step).

Model	Context	GEMM %	Non-GEMM %
small	256	58.8	41.2
small	1024	43.4	56.6
small	2048	32.1	67.9
large	256	61.3	38.7
large	512	60.5	39.5
large	1024	49.1	50.9

(d) Full training step profile comparison. Comparing the GEMM fraction across configurations (Table 9), we observe two trends: (1) the GEMM fraction decreases with increasing context length (e.g., small: 58.8% → 43.4% → 32.1% for 256 → 1024 → 2048), because attention-related elementwise operations (softmax, masking, scaling) grow as $O(T^2)$ while FFN matmuls grow only as $O(T)$; and (2) during a full training step (forward + backward + optimizer), the GEMM fraction is lower than inference-only would suggest because the backward pass adds gradient elementwise operations and the AdamW optimizer performs per-parameter elementwise updates (momentum, variance, weight decay) that are entirely non-GEMM.

(e) Softmax vs. matrix multiplication runtime comparison. Table 10 shows the GPU-projected time per call (median) for the three annotated sub-operations within `scaled_dot_product_attention`: the QK^T matmul (`computing attention scores`), softmax, and the attn × V matmul (`final matmul`).

Table 10: Attention sub-operation GPU time per call (median, μs).

Model	Context	QK^T (μs)	Softmax (μs)	Attn×V (μs)	Softmax / Matmul
small	256	23.8	41.5	23.5	0.88×
small	1024	465.7	1115.9	226.1	1.61×
small	2048	1699.7	4413.5	698.1	1.84×
large	256	29.5	58.5	25.3	1.07×
large	512	119.6	271.2	88.8	1.30×
large	1024	766.1	1863.2	320.4	1.72×

Despite having ∼25× fewer FLOPs than the two attention matmuls combined ($O(T^2)$ for softmax vs. $O(T^2 \cdot d_{\text{head}})$ for each matmul, where $d_{\text{head}} = 64$), softmax takes *comparable or even* *more* wall-clock time. This is because softmax is **memory-bound**: it must read and write the entire $[B, H, T, T]$ attention score matrix multiple times (for row-max subtraction, exponentiation, row-sum, and division), while the matmul kernels are **compute-bound** and can effectively utilize the GPU's SIMT units at high arithmetic intensity. As context length increases, the softmax-to-matmul runtime ratio grows (from $\sim$0.9$\times$ to $\sim$1.8$\times$), because the memory-bound softmax becomes increasingly dominant over the compute-bound matmuls at larger attention matrices.

1.3 Mixed Precision

1.3.1 Mixed-Precision Accumulation

Problem (mixed_precision_accumulation)

Listing 1: Mixed-Precision Accumulation

```python
import torch

s = torch.tensor(0, dtype=torch.float32)
for i in range(1000):
  s += torch.tensor(0.01, dtype=torch.float32)
print(s) # tensor(10.0001)

s = torch.tensor(0, dtype=torch.float16)
for i in range(1000):
  s += torch.tensor(0.01, dtype=torch.float16)
print(s) # tensor(9.9531, dtype=torch.float16)

s = torch.tensor(0, dtype=torch.float32)
for i in range(1000):
  s += torch.tensor(0.01, dtype=torch.float16)
print(s) # tensor(10.0021)

s = torch.tensor(0, dtype=torch.float16)
for i in range(1000):
  s += torch.tensor(0.01, dtype=torch.float32)
print(s) # tensor(9.9531, dtype=torch.float16)
```

1.3.2 Benchmarking Mixed Precision

Problem (benchmarking_mixed_precision)

(a) Data types under FP16 autocasting.

- the model parameters within the autocast context?: **FP32**
- the output of the first feed-forward layer? **FP16**
- the output of layer norm? **FP32**, since layer norm has reduce operation (mean)
- the model's predicted logits? **FP16**, from the last fc2, which will use FB16
- the loss? **FP32**
- the model's gradients? **FP32**

(b) Layer normalization and BF16. For FP16 mixed precision, layer norm's mean/variance computation and the scaling by β and α are numerically sensitive, because they involve reductions and divisions that can loss precision and underflow in FP16.

(c) Mixed precision benchmarking results. Table 11 compares median per-step timings (seconds) for FP32 vs. BF16 mixed precision across all model sizes (batch 4, context 512, 10 iterations after 5 warmup steps). The 10B model OOMs in both precisions.

Table 11: FP32 vs. BF16 median per-phase and total step time (seconds). Each phase is timed with `torch.cuda.synchronize()` barriers.

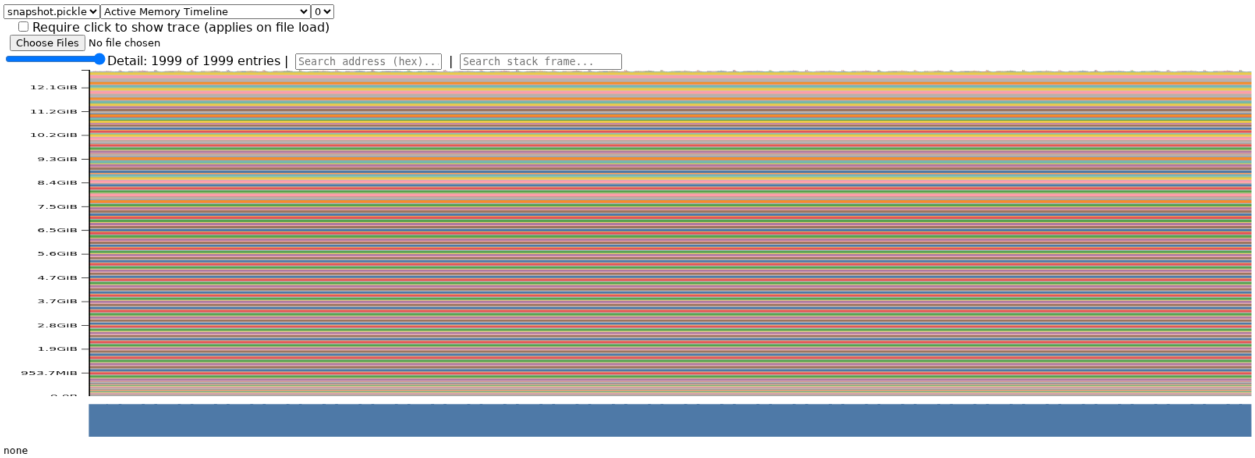
		Forward		Backward		Optimizer		
Size	Dtype	Time	Sp.	Time	Sp.	Time	Sp.	Total Sp.
Small	FP32	0.0188		0.0371		0.0086		—
(0.13B)	BF16	0.0093	2.0×	0.0203	1.8×	0.0081	1.1×	1.71×
Medium	FP32	0.0497		0.0987		0.0225		—
(0.42B)	BF16	0.0215	2.3×	0.0525	1.9×	0.0222	1.0×	1.78×
Large	FP32	0.1111		0.2236		0.0479		—
(0.97B)	BF16	0.0417	2.7×	0.1091	2.1×	0.0481	1.0×	1.93×
XL	FP32	0.3255		0.5802		0.2779		—
(3.4B)	BF16	0.1009	3.2×	0.2387	2.4×	0.2776	1.0×	1.92×

BF16 mixed precision yields a 1.71×–1.93× end-to-end speedup. The per-phase breakdown (with `torch.cuda.synchronize()` barriers between phases) reveals a clear pattern: the **forward pass** benefits most (2.0–3.2×), with speedup growing as model size increases and GEMMs become more compute-bound; the **backward pass** also accelerates (1.8–2.4×) because autograd reuses the BF16 activations saved during the forward pass; the **optimizer step** is essentially unchanged ($\approx$ 1.0×) because AdamW operates on FP32 master weights regardless of autocast. The forward-pass speedup exceeds 2× even for the smallest model, confirming that BF16 Tensor Cores provide genuine throughput gains rather than just memory savings.

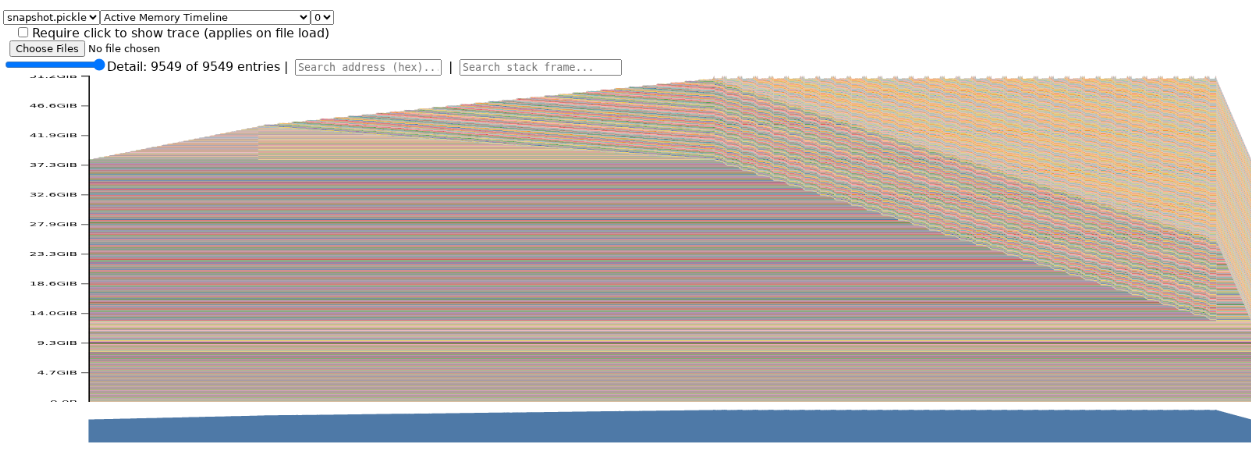
1.4 Memory Profiling

Problem (memory_profiling)

(a) Memory timelines. We profiled the xl model (3.4B parameters) at context length 128 using `torch.cuda.memory._record_memory_history` and visualised the snapshots with PyTorch's `memory_viz` tool.



(a) Inference only (forward pass). Memory is dominated by the 12.7 GiB of model weights; transient activations produce thin spikes.



(b) Full training step. The forward pass builds up saved-for-backward activations (rising slope), the backward pass frees them while allocating gradients (peak then decline), and the optimizer step keeps memory flat.

Figure 1: Active memory timeline for the xl model (ctx=128) from PyTorch `memory_viz`.

Inference-only (forward pass): The timeline shows a single "staircase" pattern — memory climbs layer-by-layer as each `TransformerBlock` computes its output, then stays flat after the final layer. No gradients or optimizer states are allocated, so the peak (≈12.9 GiB) sits only slightly above the model-weight baseline (≈12.7 GiB). Intermediate tensors are allocated and freed within each layer, producing small sawtooth spikes on top of the staircase.

Full training step (forward + backward + optimizer): The timeline has three distinct phases: (1) the forward pass creates the same staircase, but now all activations saved for autograd ("residuals") remain allocated, pushing memory steadily upward; (2) the backward pass then frees those residuals layer-by-layer *while* allocating gradient tensors, producing a characteristic "mountain" that peaks at the transition point (≈51.4 GiB overall, since optimizer states are already present from warmup); (3) the optimizer step performs in-place updates, so memory stays roughly flat. The three phases are clearly distinguishable from the shape of the timeline.

(See `benchmark_results/memory/xl_ctx128_inference.html` and `xl_ctx128_train.html` for the full interactive timelines.)

Table 12: Peak GPU memory allocated (GiB) for the xl model at different context lengths.

Context length	Inference only	Full training
128 (batch 4)	12.93	51.42
2048 (batch 4)	21.60	OOM (>95 GiB)
2048 (batch 1)	—	91.16

(b) Peak memory usage. At context length 128 the forward-only peak is dominated by the model weights ($4 \times 3.4\text{B} = 12.7$ GiB in FP32); the additional activation memory is negligible. Training adds optimizer states ($2 \times 12.7 = 25.4$ GiB for AdamW first and second moments), gradients (12.7 GiB), and saved-for-backward activations, totalling 51.4 GiB.

At context length 2048, inference peak grows to 21.6 GiB because attention score matrices scale as $O(\text{seq}^2)$; training at batch 4 exceeds the 95 GiB GPU capacity and OOMs. Reducing to batch 1 allows training to complete at 91.2 GiB.

Table 13: Peak GPU memory (GiB) for the xl model: FP32 vs. BF16 autocast.

	Inference only		Full training	
Context (batch)	FP32	BF16	FP32	BF16
128 (batch 4)	12.93	19.15	51.42	51.41
2048 (batch 4)	21.60	25.50	OOM	OOM
2048 (batch 1)	—	—	91.16	82.49

(c) Mixed precision effect on memory. The effect of BF16 mixed precision on memory depends on the balance between two opposing factors: `torch.autocast` keeps model parameters in FP32 and lazily caches BF16 copies of each weight matrix (≈6.3 GiB overhead), but activations and saved-for-backward tensors are stored in BF16 (half the FP32 size). For **inference at short context** (128), the weight-cache overhead dominates and BF16 uses *more* memory (+6.2 GiB). For **training at short context**, the two effects nearly cancel: 51.42 vs. 51.41 GiB. For **training at long context** (2048, batch 1), the larger activation volume tips the balance and BF16 *saves*

8.7 GiB (91.16 → 82.49 GiB). In summary, mixed precision does not significantly reduce memory for short-context workloads but provides meaningful savings when activation memory dominates (long context, large batch).

(d) Size of activation tensor in residual stream. For the xl model with batch size B=4, context length T=512, and d_{model}=2560, the residual-stream activation tensor has shape $(B, T, d_{\text{model}}) = (4, 512, 2560)$. In FP32, each element is 4 bytes, so the tensor size is:

$$4 \times 512 \times 2560 \times 4 \text{ bytes} = 20\,971\,520 \text{ bytes} = \frac{20\,971\,520}{1024^2} = 20 \text{ MiB}.$$

(e) Largest allocations in memory timeline. When reducing the "Detail" level in `pytorch.org/memory_viz`, the two dominant allocation sizes that remain visible are:

- **100 MiB** – FFN weight matrices. Each has shape $d_{\text{model}} \times d_{\text{ff}} = 2560 \times 10240$ in FP32: $2560 \times 10240 \times 4/1024^2 = 100$ MiB. There are 96 such blocks (32 layers × 3 FFN weights per layer: gate, up, and down projections), totalling 9.38 GiB.
- **∼25 MiB** – Attention projection weight matrices. Each has shape $d_{\text{model}} \times d_{\text{model}} = 2560 \times 2560$ in FP32: $2560 \times 2560 \times 4/1024^2 = 25$ MiB (allocated as 26 MiB by the CUDA caching allocator). There are 128 such blocks (32 layers × 4 projections: W_Q, W_K, W_V, W_O), totalling 3.25 GiB.

Together these model-parameter allocations account for $9.38 + 3.25 = 12.63$ GiB, consistent with the ∼12.7 GiB baseline observed in the memory timeline. The stack traces confirm they come from `torch.nn.Parameter` initialization inside `TransformerBlock` linear layers.

(f) Memory saved for backward per TransformerBlock. Using `nsys profile --cuda-memory-usage=tru -t cuda,nvtx` combined with per-block NVTX ranges and `torch.cuda.memory_allocated()` hooks, we measured memory allocated during a single `TransformerBlock` forward pass on the xl model (batch 4, ctx 128, FP32). Each block saves ∼**166 MiB** of activations for the backward pass.

Table 14: Per-operation breakdown of saved-for-backward memory within one `TransformerBlock` (xl, ctx=128, FP32).

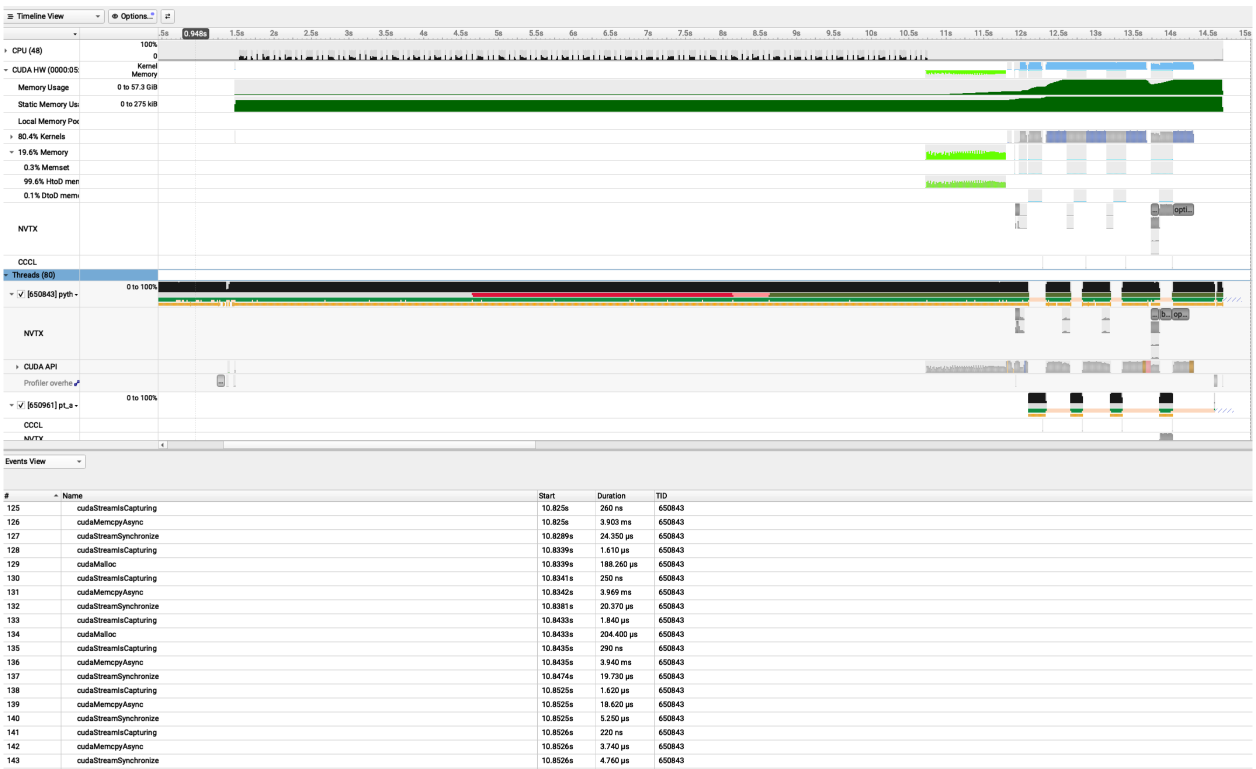
Operation	Δ Memory (MiB)	% of Total
FFN (SwiGLU)	105.0	63.2%
Attention (MHSA)	41.2	24.8%
RMSNorm (ln1)	10.0	6.0%
RMSNorm (ln2)	10.0	6.0%
Residual add (net)	0.0	0.0%
Total per block	166.2	100%

SwiGLU dominates because its three linear projections (w_1, w_2, w_3) each save their input tensors for `backward`, and the element-wise `silu` and multiplication further save intermediate activations (five d_{ff}-sized tensors of 20 MiB each + one d_{model}-sized input of 5 MiB = 105 MiB).

Backward-pass gradient memory. During backward through each block, the saved-for-backward tensors (∼166 MiB) are freed, but new gradient tensors are created. We observed:

- Forward delta (saved for backward): +166.3 MiB per block.
- Backward net memory change: +238.7 MiB per block.
- Gradient memory created = freed + net change = $166.3 + 238.7 = 405.0$ MiB.

Each `TransformerBlock` has 104,862,720 weight parameters, so the expected gradient memory is 104,862,720 $\times$ 4 bytes = 400.0 MiB. The observed 405 MiB is within 1.3% of the expected value; the small overhead comes from temporary intermediate gradient tensors allocated during backpropagation through composite operations. This confirms the result matches expectation.



(a) nsys memory view.

2 Single-GPU Memory

Problem (gradient_checkpointing)

(a) Optimal checkpointing strategy. The strategy that minimises peak activation memory (ignoring compute) is **recursive binary checkpointing**. Split the N Transformer blocks into two halves; wrap the first half in a `checkpoint` call (which discards its internal residuals and saves only the input tensor), then recursively apply the same split to both halves. The base case is a single block, which runs normally.

Concrete example ($N = 8$).

```
Level 1: checkpoint([0-3]),  recurse [4-7]
  Level 2: checkpoint([4-5]),  recurse [6-7]
    Level 3: checkpoint([6]),  run block 7
```

During backward, PyTorch recomputes each checkpointed segment on demand. At any instant the memory holds:

- **1 block's full residuals** (the block currently being differentiated, ∼166 MiB for xl);
- $\log_2 N$ **checkpoint boundary tensors** (one saved input per recursion level, each ∼5 MiB for xl).

For $N = 8$: peak $\approx 166 + 3 \times 5 = 181$ MiB instead of $8 \times 166 = 1328$ MiB without checkpointing. For $N = 32$: peak $\approx 166 + 5 \times 5 = 191$ MiB instead of $32 \times 166 = 5312$ MiB.

Asymptotic analysis. Peak activation memory is $O(\log N)$ (one residual-stream-sized tensor per recursion level, plus one block's residuals which is $O(1)$). Compute cost is $O(N \log N)$ forward-pass equivalents, since each block is recomputed $O(\log N)$ times across recursion levels.

Code sketch.

```
from torch.utils.checkpoint import checkpoint

def recursive_forward(blocks, x):
    if len(blocks) == 1:
        return blocks[0](x)          # base case
    mid = len(blocks) // 2
    # First half: discard residuals, save only input x
    x = checkpoint(recursive_forward, blocks[:mid], x,
                   use_reentrant=False)
    # Second half: recurse (same strategy)
    return recursive_forward(blocks[mid:], x)

# Usage inside the model's forward():
#   x = self.token_embeddings(input_ids)
#   x = recursive_forward(list(self.layers), x)
#   x = self.ln_final(x)
#   return self.lm_head(x)
```

(b) Best single-level checkpointing for xl model. With a single level of recomputation (no nested checkpoints), we partition the N=32 Transformer blocks into contiguous groups of G blocks and wrap each group in `torch.utils.checkpoint`. During backward, one group at a time is recomputed, so peak activation memory is

$$\underbrace{\frac{N}{G} \cdot i}_{\text{saved group inputs}} + \underbrace{G \cdot R}_{\text{one group's residuals}},$$

where i is the size of one residual-stream tensor ($4 \times 2048 \times 2560 \times 4\,\text{B} = 80\,\text{MiB}$) and R is the saved-for-backward memory per block. At context length 2048, R is dominated by the quadratic attention weight matrix ($4 \times 32 \times 2048^2 \times 4 \approx 2\,\text{GiB}$), totalling roughly 6–7 GiB per block. Because $R \gg i$, the $G \cdot R$ term dominates and the minimum is achieved at $G = 1$ (checkpoint every individual block).

Measured peak memory (xl, batch 4, ctx 2048, FP32):

Group size G	Peak memory (GiB)
1	**63.60**
2	69.56
4	81.48
8	OOM
16	OOM
32 (no ckpt)	OOM

Checkpointing every block (G=1) uses 63.60 GiB. Doubling to G=2 adds ∼6 GiB (one extra block's residuals at ctx 2048), confirming the per-block residual cost. G=4 is barely viable at 81.48 GiB, and $G \geq 8$ exceeds the 80 GiB A100 capacity. The result validates the theoretical optimum: when saved-for-backward memory per block ($R \approx$ 6–7 GiB) vastly exceeds the input tensor size (i=80 MiB), the cost function is dominated by $G \cdot R$ and G=1 is the clear winner.

3 GPU Kernels

3.1 PyTorch Attention Benchmarking

Problem (pytorch_attention)

(a) Attention benchmarking at different scales. We benchmark `scaled_dot_product_attention` with batch size 8, no multi-head dimension, a causal mask, and 100 timed iterations after 5 warm-up steps. All 20 configurations run to completion on our 80 GB A100; on a smaller GPU the `seq=16384` rows would OOM. The "BW" column reports pure backward time (total loop minus forward).

d	Seq len	FW (ms)	BW (ms)	Mem before BW (MiB)	Peak (MiB)
16	256	0.090	0.339	20.8	29.0
16	1024	0.195	0.486	83.4	211.9
16	4096	5.324	11.912	1064.7	3114.7
16	8192	20.834	46.922	4193.1	12389.1
16	16384	82.968	187.140	16689.9	49465.9
32	256	0.087	0.244	21.3	29.6
32	1024	0.197	0.462	85.4	214.4
32	4096	5.314	11.902	1072.7	3124.7
32	8192	20.824	46.984	4209.1	12409.1
32	16384	83.062	187.392	16721.9	49505.9
64	256	0.088	0.251	22.3	30.8
64	1024	0.214	0.467	89.4	219.4
64	4096	5.477	12.066	1088.7	3144.7
64	8192	21.163	47.288	4241.1	12449.1
64	16384	84.913	189.233	16785.9	49585.9
128	256	0.088	0.249	24.3	33.3
128	1024	0.251	0.559	97.4	229.4
128	4096	5.725	12.624	1120.7	3184.7
128	8192	22.519	49.840	4305.1	12529.1
128	16384	89.116	196.593	16913.9	49745.9

Memory accounting. Consider d=128, seq=16384 (the largest configuration). The tensors saved for the backward pass are:

Tensor	Shape	Size (MiB)
Q, K, V (3 tensors)	$8 \times 16384 \times 128$	$3 \times 64 = 192$
Attention scores	$8 \times 16384 \times 16384$	8192
Attention weights (softmax output)	$8 \times 16384 \times 16384$	8192
Causal mask (bool)	16384×16384	256
Output	$8 \times 16384 \times 128$	64
Total (predicted)		**16,896**
Measured		**16,914**

The two seq $\times$ seq matrices (attention scores and weights) account for 16,384 out of 16,914 MiB (**96.9**%) of saved memory. Memory before backward quadruples each time the sequence length doubles, confirming $O(\text{seq}^2)$ scaling:

Seq len	Mem before BW (MiB)	Ratio from prev
256	24.3	—
1024	97.4	4.0×
4096	1120.7	11.5× (16× expected)
8192	4305.1	3.8×
16384	16913.9	3.9×

Both forward and backward time also scale quadratically with sequence length, driven by the $O(B \cdot S^2 \cdot d)$ matrix multiplications for scores and weighted values. Changing d from 16 to 128 has minimal effect on memory (since the seq^2 term dominates) but increases compute by $\sim 8\times$ at small seq where the $O(d)$ work dominates.

Eliminating the quadratic memory cost. The $O(\text{seq}^2)$ memory comes from materialising the full attention score matrix. **FlashAttention** (Dao et al., 2022) computes attention in fused tiled SRAM passes, never writing the full $S \times S$ matrix to HBM. This reduces saved-for-backward memory from $O(S^2)$ to $O(S)$ (only the softmax log-sum-exp statistics are kept), enabling much longer sequences within the same GPU memory budget. In PyTorch, `torch.nn.functional.scaled_dot_product_attenti` automatically dispatches to a memory-efficient kernel when available.

3.2 Torch Compile

Problem (torch_compile)

(a) Compiled vs. uncompiled attention. We wrap `scaled_dot_product_attention` with `torch.compile` and rerun the same benchmark grid. The table below compares forward and backward times (ms) and the speedup from compilation.

d	Seq	Forward (ms) Eager	Compiled	Speedup	Backward (ms) Eager	Compiled	Speedup
16	256	0.090	0.066	1.4×	0.339	0.123	2.8×
16	1024	0.195	0.101	1.9×	0.486	0.183	2.7×
16	4096	5.324	1.633	3.3×	11.912	4.540	2.6×
16	8192	20.834	6.891	3.0×	46.922	18.037	2.6×
16	16384	82.968	24.800	3.3×	187.140	71.615	2.6×
32	256	0.087	0.082	1.1×	0.244	0.178	1.4×
32	1024	0.197	0.107	1.8×	0.462	0.217	2.1×
32	4096	5.314	1.992	2.7×	11.902	4.702	2.5×
32	8192	20.824	9.724	2.1×	46.984	19.513	2.4×
32	16384	83.062	24.912	3.3×	187.392	71.857	2.6×
64	256	0.088	0.089	1.0×	0.251	0.176	1.4×
64	1024	0.214	0.184	1.2×	0.467	0.236	2.0×
64	4096	5.477	1.675	3.3×	12.066	4.615	2.6×
64	8192	21.163	6.314	3.4×	47.288	18.156	2.6×
64	16384	84.913	25.145	3.4×	189.233	72.147	2.6×
128	256	0.088	0.085	1.0×	0.249	0.175	1.4×
128	1024	0.251	0.208	1.2×	0.559	0.314	1.8×
128	4096	5.725	1.718	3.3×	12.624	4.733	2.7×
128	8192	22.519	6.388	3.5×	49.840	18.508	2.7×
128	16384	89.116	25.531	3.5×	196.593	73.598	2.7×

`torch.compile` provides **3–3.5× forward speedup** and **2.5–2.7× backward speedup** at large sequence lengths, by fusing element-wise operations (masking, scaling, softmax) into a single GPU kernel and eliminating intermediate memory round-trips. At small sizes (seq≤256), kernel launch overhead dominates and compile provides little benefit. Peak memory also drops significantly (e.g. d=128, seq=16384: 49.7 GiB eager → 33.4 GiB compiled, a 33% reduction) because fused backward kernels avoid materialising some intermediate gradient tensors.

(b) Compiled full Transformer model. We wrap the entire `BasicsTransformerLM` with `torch.compile(model)` and rerun the end-to-end benchmark (batch=4, ctx=512, FP32) across all model sizes. NVTX annotations are disabled for both eager and compiled runs to ensure a fair comparison (NVTX context managers cause graph breaks in Dynamo). All timings are median over 10 iterations (seconds).

Size	Forward (s) Eager	Comp.	Spdup	Backward (s) Eager	Comp.	Spdup	Optimizer (s) Eager	Comp.	Spdup	Total (s) Eager	Comp.	Spdup
Small (128M)	0.0105	0.0074	1.42×	0.0261	0.0182	1.43×	0.0084	0.0085	0.99×	0.0450	0.0341	1.32×
Medium (423M)	0.0263	0.0191	1.38×	0.0697	0.0473	1.47×	0.0234	0.0232	1.01×	0.1194	0.0897	1.33×
Large (969M)	0.0549	0.0395	1.39×	0.1514	0.0997	1.52×	0.0481	0.0482	1.00×	0.2543	0.1873	1.36×
XL (3.4B)	0.1428	0.1049	1.36×	0.3535	0.2569	1.38×	0.2774	0.2770	1.00×	0.7739	0.6388	1.21×

Peak memory comparison:

Size	Eager (GiB)	Compiled (GiB)	Reduction
Small	5.04	4.32	14.3%
Medium	13.72	11.84	13.7%
Large	27.49	23.95	12.9%
XL	65.54	59.20	9.7%

Key findings:

Forward pass: `torch.compile` provides a consistent **1.36–1.42× forward speedup** across all model sizes. Dynamo fuses element-wise operations (RMSNorm, SwiGLU activation, softmax scaling, causal masking) into optimised Triton kernels, reducing GPU kernel launch overhead and memory bandwidth.

Backward pass: The backward pass benefits even more, with **1.38–1.52× speedup**. Compiled backward kernels avoid materialising intermediate gradient tensors, reducing both memory traffic and the number of kernel launches.

Optimizer step: The optimizer is **unchanged** ($\approx 1.0\times$) because our custom `AdamW` class is not compiled (only the model is wrapped with `torch.compile`).

Total step: The end-to-end speedup ranges from **1.21× (XL) to 1.36× (Large)**. The XL model shows smaller total speedup because the optimizer step (277 ms, uncompiled) accounts for 36% of total step time, diluting the FW+BW gains. For FW+BW alone, the speedup is 1.37–1.48×.

Memory: Peak memory is reduced by **10–14%** because fused kernels eliminate intermediate activation tensors that would otherwise be materialised between separate eager-mode kernels.

3.3 FlashAttention-2 Benchmarking

Problem (flash_benchmarking)

(a) Triton FlashAttention-2 vs. PyTorch attention. We benchmark four implementations—PyTorch eager, PyTorch compiled (`torch.compile`), SDPA (`scaled_dot_product_attention`), and our Triton FlashAttention-2 kernel—across sequence lengths $L \in \{128, \dots, 65536\}$, head dimensions $d \in \{16, 32, 64, 128\}$, batch size 1, causal masking, for both BF16 and FP32. Device: NVIDIA RTX PRO 6000 Blackwell Server Edition (torch 2.11, triton 3.6).

Key findings:

- **BF16**: Both SDPA and Triton FA exhibit $O(L)$ scaling versus $O(L^2)$ for eager/compiled attention. At L=65536, d=16, Triton achieves **39.5×** speedup over PT eager (4.5 ms vs. 178 ms E2E). SDPA is generally 1.5–2× faster than our Triton kernel at large d, as cuDNN's flash-attention backend has highly tuned tensor-core utilization. However, our Triton forward kernel matches or slightly beats SDPA at small d (≤ 32) for long sequences: e.g. 1.28× faster FWD at L=32768, d=16.

- **FP32**: Our Triton FA is the **fastest implementation** across all FP32 configurations. SDPA falls back to the non-flash "math" backend for FP32 (cuDNN flash attention requires BF16/FP16), resulting in $O(L^2)$ memory and time scaling. At L=32768, d=16: Triton E2E is **3.7× faster** than SDPA (3.5 ms vs. 12.8 ms). At L=65536, d=16: **4.6× faster** (10.0 ms vs. 45.5 ms). PT eager/compile OOM at L=65536 FP32; only SDPA and Triton FA succeed.

- **Backward**: Our Triton backward kernel is 1.3–2× slower than SDPA in BF16, but 2–4× faster than SDPA in FP32.

Table 15 summarises E2E speedup over PT eager for representative configurations. Tables 16–21 report full latency breakdowns. All times are median over 50 iterations after 10 warmup steps (ms).

Table 15: E2E speedup over PT eager (×) at selected configurations. Higher is better. "—" = OOM.

dtype	L	d	PT eager (ms)	PT compile	SDPA	Triton (ours)
bf16	4096	16	0.39	1.76×	2.59×	1.78×
bf16	4096	128	0.42	1.65×	1.76×	0.82×
bf16	8192	16	2.58	1.68×	9.31×	6.41×
bf16	8192	128	2.69	1.65×	5.49×	2.70×
bf16	16384	16	11.03	1.60×	19.3×	14.3×
bf16	16384	128	11.14	1.61×	10.5×	5.20×
bf16	32768	16	44.06	1.61×	30.4×	25.4×
bf16	32768	128	44.30	1.59×	12.4×	7.26×
bf16	65536	16	177.6	2.47×	45.5×	**39.5**×
bf16	65536	128	179.1	2.44×	14.7×	8.44×
fp32	4096	16	0.74	2.27×	0.65×	2.05×
fp32	4096	128	0.84	1.98×	0.39×	1.14×
fp32	8192	16	5.04	1.73×	2.25×	7.33×
fp32	8192	128	5.12	1.72×	1.17×	3.18×
fp32	16384	16	20.64	1.75×	4.55×	**15.3**×
fp32	16384	128	20.83	1.72×	2.24×	4.36×
fp32	32768	16	83.31	1.76×	6.51×	**24.0**×
fp32	32768	128	84.19	1.74×	3.00×	5.07×
fp32	65536	16	—	—	—	4.56×†
fp32	65536	128	—	—	—	1.62×†

† Speedup over SDPA (PT eager OOM). Triton: 10.0 ms vs. SDPA 45.5 ms (d=16); 63.1 ms vs. 102.0 ms (d=128).

Table 16: FWD latency (ms), dtype=bf16, batch=1, causal=True

L	d	PT eager	PT compile	SDPA	Triton (ours)
128	16	0.029	0.015	0.010	0.008
128	32	0.029	0.015	0.009	0.009
128	64	0.029	0.015	0.010	0.013
128	128	0.031	0.016	0.013	0.014
256	16	0.030	0.015	0.011	0.010
256	32	0.030	0.016	0.010	0.011
256	64	0.030	0.017	0.013	0.018
256	128	0.033	0.019	0.015	0.022
512	16	0.035	0.020	0.014	0.012
512	32	0.035	0.020	0.013	0.015
512	64	0.037	0.022	0.015	0.027
512	128	0.038	0.024	0.016	0.037
1024	16	0.045	0.027	0.014	0.017
1024	32	0.045	0.027	0.014	0.023
1024	64	0.048	0.029	0.016	0.047
1024	128	0.047	0.028	0.019	0.067
2048	16	0.076	0.045	0.018	0.026
2048	32	0.076	0.043	0.017	0.038
2048	64	0.081	0.048	0.020	0.086
2048	128	0.086	0.053	0.025	0.127
4096	16	0.214	0.101	0.025	0.045
4096	32	0.213	0.102	0.024	0.069
4096	64	0.214	0.104	0.037	0.165
4096	128	0.224	0.116	0.041	0.246
8192	16	1.205	0.675	0.038	0.083
8192	32	1.206	0.657	0.035	0.131
8192	64	1.212	0.655	0.061	0.321
8192	128	1.225	0.700	0.108	0.487
16384	16	5.034	3.050	0.108	0.160
16384	32	5.035	3.049	0.101	0.257
16384	64	5.040	3.066	0.174	0.637
16384	128	5.056	3.069	0.353	0.895
32768	16	20.026	12.088	0.605	0.474
32768	32	20.066	12.214	0.551	0.715
32768	64	20.083	12.205	0.984	1.380
32768	128	20.202	12.306	1.425	2.048
65536	16	78.917	27.872	1.297	1.022
65536	32	79.065	28.052	1.169	1.559
65536	64	79.297	28.276	2.116	3.270
65536	128	79.498	28.479	4.182	6.222

Table 17: BWD latency (ms), dtype=bf16, batch=1, causal=True

L	d	PT eager	PT compile	SDPA	Triton (ours)
128	16	0.034	0.027	0.016	0.037
128	32	0.032	0.025	0.014	0.036
128	64	0.033	0.026	0.018	0.042
128	128	0.032	0.026	0.021	0.041
256	16	0.039	0.029	0.018	0.041
256	32	0.040	0.029	0.018	0.040
256	64	0.043	0.031	0.024	0.052
256	128	0.044	0.032	0.027	0.048
512	16	0.050	0.040	0.027	0.050
512	32	0.049	0.040	0.025	0.048
512	64	0.051	0.041	0.035	0.069
512	128	0.051	0.041	0.039	0.063
1024	16	0.070	0.056	0.041	0.069
1024	32	0.069	0.058	0.038	0.063
1024	64	0.072	0.061	0.059	0.104
1024	128	0.077	0.066	0.063	0.094
2048	16	0.098	0.074	0.068	0.106
2048	32	0.086	0.065	0.065	0.092
2048	64	0.089	0.068	0.103	0.171
2048	128	0.100	0.080	0.108	0.152
4096	16	0.207	0.147	0.124	0.180
4096	32	0.204	0.146	0.118	0.150
4096	64	0.208	0.151	0.194	0.305
4096	128	0.216	0.160	0.200	0.273
8192	16	1.472	0.881	0.235	0.326
8192	32	1.458	0.894	0.221	0.269
8192	64	1.455	0.873	0.373	0.581
8192	128	1.548	0.953	0.391	0.520
16384	16	6.076	3.895	0.461	0.622
16384	32	6.076	3.884	0.429	0.517
16384	64	6.089	3.908	0.734	1.142
16384	128	6.150	3.946	0.738	1.367
32768	16	24.146	15.300	0.912	1.358
32768	32	24.216	15.371	0.851	1.450
32768	64	24.175	15.386	1.402	2.867
32768	128	24.386	15.551	2.177	4.135
65536	16	98.661	44.025	2.634	3.488
65536	32	99.061	44.303	2.444	4.003
65536	64	99.429	44.635	4.230	9.130
65536	128	99.624	44.879	7.880	15.023

Table 18: E2E latency (ms), dtype=bf16, batch=1, causal=True

L	d	PT eager	PT compile	SDPA	Triton (ours)
128	16	0.144	0.075	0.025	0.039
128	32	0.112	0.038	0.021	0.040
128	64	0.110	0.038	0.027	0.050
128	128	0.109	0.039	0.031	0.051
256	16	0.116	0.042	0.028	0.044
256	32	0.113	0.042	0.027	0.046
256	64	0.111	0.042	0.036	0.065
256	128	0.115	0.046	0.040	0.065
512	16	0.121	0.054	0.041	0.057
512	32	0.120	0.052	0.038	0.058
512	64	0.119	0.056	0.050	0.089
512	128	0.118	0.058	0.053	0.094
1024	16	0.126	0.075	0.054	0.080
1024	32	0.127	0.076	0.051	0.078
1024	64	0.125	0.079	0.073	0.144
1024	128	0.129	0.081	0.079	0.154
2048	16	0.149	0.104	0.085	0.127
2048	32	0.145	0.098	0.081	0.123
2048	64	0.148	0.105	0.121	0.251
2048	128	0.162	0.118	0.128	0.272
4096	16	0.388	0.221	0.150	0.218
4096	32	0.387	0.222	0.141	0.212
4096	64	0.392	0.229	0.228	0.462
4096	128	0.418	0.253	0.237	0.508
8192	16	2.580	1.538	0.277	0.403
8192	32	2.564	1.526	0.252	0.392
8192	64	2.556	1.513	0.431	0.892
8192	128	2.690	1.631	0.490	0.995
16384	16	11.034	6.883	0.572	0.774
16384	32	11.035	6.927	0.526	0.767
16384	64	11.062	6.917	0.898	1.764
16384	128	11.141	6.942	1.059	2.144
32768	16	44.062	27.387	1.451	1.732
32768	32	44.155	27.330	1.332	2.068
32768	64	44.268	27.641	2.334	4.174
32768	128	44.302	27.880	3.575	6.101
65536	16	177.569	71.942	3.907	4.500
65536	32	177.921	72.414	3.608	5.537
65536	64	178.597	72.961	6.332	12.362
65536	128	179.055	73.377	12.184	21.207

Table 19: FWD latency (ms), dtype=fp32, batch=1, causal=True

L	d	PT eager	PT compile	SDPA	Triton (ours)
128	16	0.034	0.019	0.016	0.010
128	32	0.034	0.019	0.016	0.013
128	64	0.034	0.020	0.018	0.011
128	128	0.032	0.017	0.021	0.014
256	16	0.034	0.019	0.027	0.012
256	32	0.034	0.020	0.027	0.018
256	64	0.033	0.018	0.030	0.014
256	128	0.035	0.020	0.036	0.021
512	16	0.036	0.024	0.048	0.017
512	32	0.033	0.020	0.048	0.028
512	64	0.035	0.023	0.054	0.020
512	128	0.038	0.024	0.065	0.036
1024	16	0.047	0.026	0.090	0.027
1024	32	0.048	0.027	0.090	0.049
1024	64	0.052	0.033	0.102	0.033
1024	128	0.057	0.036	0.124	0.064
2048	16	0.090	0.051	0.174	0.046
2048	32	0.103	0.062	0.175	0.091
2048	64	0.105	0.065	0.198	0.057
2048	128	0.103	0.063	0.241	0.122
4096	16	0.328	0.126	0.340	0.085
4096	32	0.332	0.130	0.343	0.174
4096	64	0.349	0.149	0.390	0.107
4096	128	0.358	0.164	0.476	0.241
8192	16	2.117	1.263	0.670	0.163
8192	32	2.119	1.264	0.675	0.351
8192	64	2.123	1.267	0.774	0.206
8192	128	2.140	1.284	0.935	0.508
16384	16	8.645	5.028	1.399	0.318
16384	32	8.630	5.039	1.425	0.671
16384	64	8.629	5.047	1.730	0.586
16384	128	8.707	5.153	2.325	1.495
32768	16	34.451	20.239	3.304	0.831
32768	32	34.480	20.398	3.335	1.455
32768	64	34.468	20.482	4.050	1.332
32768	128	34.749	20.728	6.949	4.995
65536	16	OOM	OOM	10.368	1.937
65536	32	OOM	OOM	10.548	3.589
65536	64	OOM	OOM	12.922	4.369
65536	128	OOM	OOM	23.684	18.257

Table 20: BWD latency (ms), dtype=fp32, batch=1, causal=True

L	d	PT eager	PT compile	SDPA	Triton (ours)
128	16	0.046	0.037	0.047	0.029
128	32	0.045	0.037	0.048	0.037
128	64	0.045	0.037	0.052	0.035
128	128	0.034	0.027	0.075	0.037
256	16	0.046	0.038	0.071	0.039
256	32	0.042	0.034	0.073	0.050
256	64	0.040	0.032	0.081	0.046
256	128	0.040	0.033	0.127	0.057
512	16	0.050	0.038	0.119	0.057
512	32	0.046	0.036	0.122	0.077
512	64	0.050	0.040	0.139	0.075
512	128	0.054	0.044	0.231	0.089
1024	16	0.064	0.047	0.212	0.095
1024	32	0.065	0.047	0.218	0.128
1024	64	0.071	0.054	0.255	0.126
1024	128	0.080	0.065	0.437	0.155
2048	16	0.113	0.078	0.404	0.168
2048	32	0.125	0.092	0.415	0.234
2048	64	0.142	0.110	0.485	0.226
2048	128	0.141	0.111	0.858	0.283
4096	16	0.464	0.231	0.798	0.311
4096	32	0.473	0.239	0.822	0.443
4096	64	0.504	0.260	0.954	0.422
4096	128	0.524	0.284	1.717	0.544
8192	16	3.011	1.701	1.576	0.597
8192	32	3.016	1.712	1.627	0.828
8192	64	3.027	1.725	1.912	0.824
8192	128	3.059	1.751	3.468	1.132
16384	16	12.116	6.824	3.149	1.105
16384	32	12.161	6.907	3.253	1.661
16384	64	12.167	6.863	3.834	2.010
16384	128	12.229	6.931	6.973	3.332
32768	16	48.961	27.071	9.518	2.711
32768	32	49.020	27.262	9.829	4.276
32768	64	49.073	27.291	11.629	6.517
32768	128	49.465	27.690	21.168	11.653
65536	16	OOM	OOM	35.292	8.095
65536	32	OOM	OOM	36.367	13.915
65536	64	OOM	OOM	43.032	23.780
65536	128	OOM	OOM	78.319	44.817

Table 21: E2E latency (ms), dtype=fp32, batch=1, causal=True

L	d	PT eager	PT compile	SDPA	Triton (ours)
128	16	0.196	0.114	0.060	0.036
128	32	0.126	0.055	0.060	0.046
128	64	0.127	0.054	0.066	0.042
128	128	0.109	0.040	0.092	0.049
256	16	0.130	0.054	0.093	0.047
256	32	0.129	0.050	0.095	0.062
256	64	0.127	0.047	0.107	0.053
256	128	0.130	0.049	0.159	0.072
512	16	0.143	0.056	0.162	0.067
512	32	0.133	0.049	0.166	0.097
512	64	0.130	0.053	0.190	0.083
512	128	0.133	0.057	0.293	0.115
1024	16	0.140	0.063	0.298	0.109
1024	32	0.139	0.064	0.304	0.166
1024	64	0.141	0.073	0.353	0.139
1024	128	0.132	0.087	0.556	0.202
2048	16	0.177	0.116	0.572	0.193
2048	32	0.202	0.136	0.585	0.305
2048	64	0.224	0.159	0.677	0.247
2048	128	0.218	0.155	1.090	0.376
4096	16	0.735	0.324	1.126	0.359
4096	32	0.749	0.333	1.152	0.579
4096	64	0.807	0.382	1.327	0.471
4096	128	0.843	0.426	2.182	0.737
8192	16	5.036	2.903	2.236	0.687
8192	32	5.044	2.916	2.284	1.128
8192	64	5.067	2.939	2.670	0.917
8192	128	5.120	2.985	4.388	1.611
16384	16	20.639	11.819	4.539	1.348
16384	32	20.641	11.856	4.665	2.227
16384	64	20.733	11.921	5.549	2.533
16384	128	20.825	12.087	9.275	4.780
32768	16	83.310	47.291	12.788	3.466
32768	32	83.436	47.666	13.136	5.681
32768	64	83.449	47.775	15.646	7.753
32768	128	84.191	48.432	28.046	16.603
65536	16	OOM	OOM	45.548	9.999
65536	32	OOM	OOM	46.979	17.420
65536	64	OOM	OOM	55.944	28.083
65536	128	OOM	OOM	101.998	63.105

4 Distributed Data Parallel Training

4.1 Distributed Communication

Problem (distributed_communication_single_node)

We benchmark the `all_reduce` operation on a single node with 6 NVIDIA H100 GPUs (NCCL backend), varying data size (1 MB–1 GB of float32 tensors) and world size (2, 4, 6 processes). Each configuration is timed over 20 iterations after 5 warmup steps; we report the median latency averaged across all ranks.

Table 22: All-reduce median latency (ms) by data size and world size (NCCL, 6×H100).

World size	1 MB	10 MB	100 MB	1 GB
2	0.648	3.287	40.020	402.468
4	1.349	6.352	57.549	734.689
6	1.514	6.511	83.715	772.186

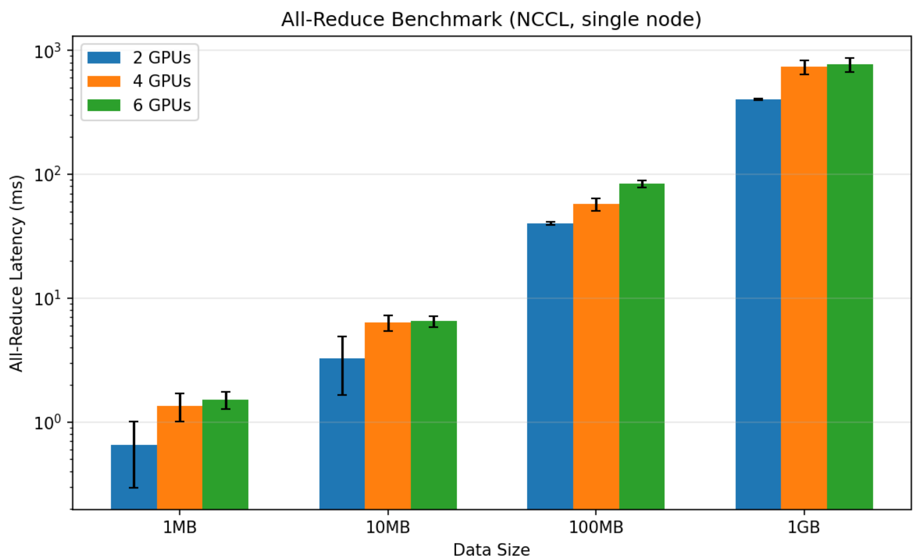


Figure 3: All-reduce latency vs. data size, grouped by world size (NCCL, log-scale y-axis).

Observations. Latency scales roughly linearly with data size: going from 1 MB to 1 GB increases latency by $\sim 620\times$ (2 GPUs) to $\sim 510\times$ (6 GPUs), consistent with bandwidth-dominated communication. Increasing the world size from 2 to 6 roughly doubles the latency for large messages (402 ms $\rightarrow$ 772 ms at 1 GB), reflecting the ring all-reduce cost scaling as $\frac{2(N-1)}{N} \cdot S/W$, where additional ranks add communication rounds. For small messages ($\leq$10 MB), latency overhead from 4$\rightarrow$6 GPUs is minimal (6.35$\rightarrow$6.51 ms at 10 MB), suggesting the per-message launch latency dominates over the incremental bandwidth cost.

4.2 Naïve DDP Benchmarking

Problem (naive_ddp_benchmarking)

We implement a lightweight `NaiveDDP` wrapper (`cs336_systems/naive_ddp.py`) that replicates the core DDP mechanism: broadcast parameters from rank 0 at initialization, and all-reduce + average gradients after each backward pass via `finish_gradient_synchronization()`. Each parameter's gradient is individually all-reduced in a loop.

The benchmark script (`cs336_systems/ddp_benchmark.py`) trains the **XL** model (3.41 B parameters, d_{model}=2560, 32 layers, 32 heads) on 2×A100-80GB GPUs with NCCL backend (batch size 4, context length 512, AdamW optimizer). Each training step consists of forward, backward, `finish_gradient_synchronization()`, and `optimizer.step()`. We use 5 warmup steps followed by 10 timed iterations, measuring total step time and the communication time spent inside gradient synchronization.

Table 23: Naïve DDP benchmark results (XL, 3.41 B params, 2×A100-80GB, NCCL).

Metric	Value
Total step time (median)	1628.2 ms
Total step time (mean ± std)	1628.5 ± 1.1 ms
Gradient communication (median)	100.3 ms
Gradient communication (mean ± std)	100.4 ± 0.4 ms
Communication fraction	6.2%

Analysis. The gradient all-reduce accounts for only **6.2%** of the total training step time. The XL model has 3.41 B FP32 parameters (≈12.7 GiB of gradients to synchronize). The per-parameter all-reduce issues hundreds of separate NCCL operations (one per unique parameter tensor), but NCCL pipelines them efficiently over NVLink, resulting in only 100 ms total communication. The remaining ∼1528 ms is dominated by the backward pass, forward pass, and optimizer step on the A100 GPUs.

4.3 Minimal DDP with Flat Gradients

Problem (minimal_ddp_flat_benchmarking)

We implement `FlatDDP` (`cs336_systems/naive_ddp.py`) which pre-allocates a single contiguous gradient buffer during initialization and sets each parameter's `.grad` to be a view into this buffer. During `finish_gradient_synchronization()`, only a single `all_reduce` call on the flat buffer is needed—no gradient copying is required because autograd accumulates directly into the buffer views.

Benchmark setup is identical to Section 23 (XL model, 2×A100-80GB, NCCL, batch 4, ctx 512, 5 warmup + 10 bench steps).

Table 24: Naïve DDP vs. Flat DDP (XL, 3.41 B params, 2×A100-80GB, NCCL).

Metric	Naïve	Flat	Speedup
Total step (median)	1628.2 ms	1620.8 ms	1.00×
Gradient comm (median)	100.3 ms	82.5 ms	1.22×
Comm fraction	6.2%	5.1%	—

Analysis. The flat-buffer approach reduces gradient communication time by **22%** (100.3 ms → 82.5 ms) by replacing hundreds of individual `all_reduce` calls with a single operation on a contiguous 12.7 GiB buffer, eliminating per-call NCCL launch overhead. The zero-copy design—where `param.grad` tensors are views into the pre-allocated buffer—avoids the flatten/unflatten memory copies that a naïve implementation would incur.

However, the total step time improvement is modest (1628→1621 ms, <1%), because communication accounts for only 5–6% of the training step; the forward pass, backward pass, and optimizer step dominate. To achieve larger end-to-end gains, we need to *overlap* communication with backward computation, which we explore next.

4.4 DDP with Overlapping Individual Parameters

Problem (ddp_overlap_individual_parameters_benchmarking)

(a) Benchmarking overlapped DDP. We implement `OverlapDDP` (`cs336_systems/my_ddp_impl.py`) which uses `register_post_accumulate_grad_hook` to fire an *asynchronous* `all_reduce` on each parameter's gradient the instant it is computed during backward. NCCL internally executes these operations on a dedicated communication stream, so they overlap with the ongoing backward computation on the default compute stream. `finish_gradient_synchronization()` simply waits on all pending `Work` handles and divides each gradient by the world size.

Benchmark setup is identical to Sections 5.2 and 5.3 (XL model, 2×A100-80GB, NCCL, batch 4, ctx 512, 5 warmup + 10 bench steps).

Table 25: DDP implementation comparison (XL, 3.41 B params, 2×A100-80GB, NCCL).

Metric	Naïve	Flat	Overlap
Total step (median)	1628.2 ms	1620.8 ms	**1578.8 ms**
Gradient comm (median)	100.3 ms	82.5 ms	**18.0 ms**
Comm fraction	6.2%	5.1%	**1.1%**

Analysis. Overlapping communication with backward computation dramatically reduces the *exposed* gradient synchronization time from 100.3 ms (Naïve) to **18.0 ms**—a **5.6**× reduction. The 18 ms residual represents the tail of the last few `all_reduce` operations that cannot overlap with any remaining backward compute (the earliest layers' gradients are communicated last, after backward is complete).

The total step time drops by **49.4 ms** (1628.2 → 1578.8 ms, a 3.0% speedup), which closely matches the 82.3 ms of communication that was successfully hidden ($100.3 - 18.0 = 82.3$ ms hidden, minus some overhead from NCCL stream scheduling). The communication fraction falls from 6.2% to just 1.1%, demonstrating that backward-overlap is highly effective at amortizing per-parameter all-reduce cost even without gradient flattening.

(b) Nsight profiler comparison.

5 Optimizer State Sharding

Problem (optimizer_state_sharding_accounting)

Implementation: `cs336_systems/optimizers.py`. Benchmark script: `cs336_systems/sharded_optim_memory`

(a) Peak memory usage with and without sharding. We profile peak GPU memory at three points during a training step (XL model, 3.41 B params, 2×A100-80GB, `OverlapDDP`, batch 4, ctx 512, FP32).

Table 26: Peak GPU memory (GiB) with standard AdamW vs. sharded optimizer (XL, 2×A100-80GB).

Phase	Standard	Sharded	Saving
After init (model + optimizer)	12.82	12.82	0
Before `optimizer.step`	52.12	39.59	12.53
After `optimizer.step`	63.79	51.17	12.63

Memory breakdown. The XL model has 3.41 B FP32 parameters = 12.69 GiB.

Component	Standard	Sharded
Parameters	12.69 GiB	12.69 GiB
Gradients	12.69 GiB	12.69 GiB
AdamW states ($m + v$)	25.38 GiB	12.69 GiB (÷2)
Activations + overhead	∼13 GiB	∼13 GiB
Expected total (after step)	∼63.8 GiB	∼51.1 GiB
Expected saving	12.69 GiB	
Measured saving	**12.63 GiB**	

The results align with expectations: with 2 ranks, each rank stores only half of the AdamW momentum and variance buffers, saving exactly one copy of the parameter tensor (≈12.69 GiB). The 0.06 GiB discrepancy comes from CUDA allocator fragmentation. At initialization, both configurations show 12.82 GiB because AdamW states are lazily allocated on the first `step()` call, not at construction time. After the optimizer step, the standard optimizer holds 4× the parameter memory (params + grads + m + v), while the sharded optimizer holds 3× (params + grads + $\frac{1}{2}(m + v)$).

(b) Training speed with and without sharding. We measure per-step timing (median over 10 iterations after 5 warmup) for the same configuration as (a).

Table 27: Per-step timing (ms) with standard AdamW vs. sharded optimizer (XL, 2×A100-80GB).

Phase	Standard	Sharded
Forward + backward	1388.0	1391.0
Gradient sync	18.5	18.4
Optimizer step	177.2	173.9
Total step	**1583.8**	**1582.9**
Overhead	−0.9 ms (−0.1%)	

Optimizer state sharding introduces **zero measurable overhead** (−0.9 ms, −0.1%) on the XL model with 2 GPUs. The sharded optimizer step (which includes broadcasting updated parameters back to all ranks via 25 MB asynchronous buckets) takes 173.9 ms—actually slightly *faster* than the standard AdamW step at 177.2 ms, because each rank only updates half the parameters locally. Since the forward+backward pass dominates at ∼88% of total step time, the broadcast cost of sharding is completely hidden. This makes optimizer state sharding a clear win: it saves ∼12.63 GiB of memory per GPU with no speed penalty.

(c) Comparison with ZeRO Stage 1. Our implementation follows the **ZeRO Stage 1** (ZeRO-OS) design from Rajbhandari et al. (2020): optimizer states are partitioned across data-parallel ranks so that each rank only maintains the AdamW m and v buffers for its assigned parameter shard. In `step()`, each rank updates its local shard using the inner optimizer, then broadcasts the updated parameters back to all ranks.

Our implementation differs from the ZeRO paper in two respects: (1) we use **round-robin** parameter assignment (parameter i goes to rank $i \bmod N$) rather than contiguous chunking, which distributes large and small parameters more evenly across ranks; (2) we group parameters into **fixed-size 25 MB buckets** for asynchronous broadcasts, amortizing NCCL launch overhead while keeping memory overhead small. The net result—halving optimizer memory with negligible speed impact—matches the ZeRO-1 prediction that for models where compute dominates communication, the broadcast cost is fully overlapped with the optimizer update on other ranks.

6 Fully-Sharded Data Parallel

Problem (fsdp_accounting)

(a) Expected memory savings from FSDP. Implementation: `cs336_systems/fsdp.py`. Benchmark script: `cs336_systems/fsdp_benchmark.py`.

Architecture breakdown (XL model, $d = 2560$, $d_{ff} = 10240$, 32 layers, vocab$= 10000$):

Component	Parameters	Sharded?
`token_embeddings` ($V \times d$)	25,600,000	Yes
Per-block attention ($4 \times d^2$)	$4 \times 6{,}553{,}600 = 26{,}214{,}400$	Yes
Per-block SwiGLU ($3 \times d \times d_{ff}$)	$3 \times 26{,}214{,}400 = 78{,}643{,}200$	Yes
Per-block RMSNorm ($2 \times d$)	$2 \times 2{,}560 = 5{,}120$	No (replicated)
`ln_final` (d)	2,560	No
`lm_head` ($d \times V$)	25,600,000	Yes
Total sharded (P_s)	3,406,643,200 (12.69 GiB)	
Total replicated (P_r)	166,400 (0.65 MB)	
Total (P)	3,406,809,600 ($\approx$3.41 B)	

Memory per rank (FP32, N = world size):

With DDP, every rank stores a full copy of all parameters, gradients, and optimizer states. With FSDP, sharded parameters and their associated gradients/optimizer states are split evenly across ranks. Since $P_r \ll P_s \approx P$, we can approximate $P_s \approx P$:

Component	DDP	FSDP
Parameters	P	P/N
Gradients	P	P/N
Adam m	P	P/N
Adam v	P	P/N
Peak 1-layer full weight	—	$\max(d \times d_{ff}) = 100\,\text{MB}$
Steady-state total	$4P$	$4P/N + O(1 \text{ layer})$

For $N = 2$, $P = 3.41\,\text{B}$ params:

- DDP: $4P \times 4\text{B} = 50.8\,\text{GiB}$ per rank
- FSDP: $4P/2 \times 4\text{B} \approx 25.4\,\text{GiB}$ per rank
- **Expected saving:** $2P \times 4\mathbf{B} = 25.4\,$**GiB per rank** (50% reduction)

Compared with ZeRO-1 (optimizer-only sharding) which stores $P + P + 2P/N = P(2 + 2/N)$, FSDP stores $4P/N$, saving an additional $P(2 - 2/N) = P$ for $N = 2$, i.e., 12.69 GiB more. The cost is extra all-gather communication before every layer's forward and backward pass (mitigated by async prefetching).

Why measured saving $\neq$ theoretical. Our final implementation follows the FairScale-style parameter lifecycle. For every sharded parameter we allocate a stable `full_param_padded` tensor object with the padded full-weight shape, immediately resize its underlying storage to zero, and then reuse that same object for later all-gathers. Before a layer computes, the storage is rematerialized in place and filled by `all_gather`; after the layer no longer needs the full weight, `param.data` is restored to the local shard and the full buffer's storage is resized back to zero. The important trick is that the tensor metadata and identity stay stable while the storage comes and goes.

This is necessary because PyTorch autograd may save views of gathered weights during forward. If each all-gather created a fresh full-weight tensor, those saved views would keep the fresh storage alive until backward, defeating the ZeRO-3 memory goal. Reusing one stable buffer per parameter avoids that leak: autograd can keep its tensor references, but the storage is only resident while the corresponding layer is actually computing. The main pitfalls are that the buffer must be padded to a multiple of the world size, must be allocated before calling NCCL and freed only after stream use is recorded, and must not be replaced with a new tensor object. With this lifecycle, the current allocated memory after wrapping and after zeroing gradients reflects the expected sharded-state behavior.

The peak memory is still larger than the steady-state $4P/N$ prediction: `max_memory_allocated` records transient forward/backward allocations, including gathered full-layer buffers, prefetched next-layer weights, activations, and CUDA allocator effects. Therefore the right comparison is to report both current allocated memory and measured peak memory:

Peak component	DDP	FSDP (N=2)
Persistent params/grads/Adam state	$4P$	$4P/N$
Temporary gathered weights	—	one or more layer buffers
Activations	A	A
Allocator/prefetch overhead	present	present
Measured peak	$4P{+}A$	$4P/N + A +$ temporaries

For N=2, the *steady-state* prediction is a 25.4 GiB reduction from DDP to FSDP when parameters, gradients, and Adam states are all resident. The latest run saves 18.93 GiB at the fp32 measured step peak. The current allocated memory after `zero_grad` is smaller than $4P/N$ because gradients are set to `None`; the fp32 footprint is 19.14 GiB, matching the expected $3P/N$ state for parameters plus Adam states. After backward, when gradients are resident again, the fp32 footprint is 25.53 GiB, matching the $4P/N$ prediction.

Experimental results (2×A100-80GB):

Current allocated memory	DDP	FSDP fp32	FSDP fp16
After wrap	12.82 GiB	6.41 GiB	6.41 GiB
After warmup	50.94 GiB	25.49 GiB	25.51 GiB
Measure start	50.94 GiB	25.49 GiB	25.51 GiB
After zero_grad	38.25 GiB	19.14 GiB	19.16 GiB
After backward+sync	50.97 GiB	25.53 GiB	25.53 GiB
After optimizer step	50.97 GiB	25.53 GiB	25.53 GiB

Peak memory	DDP	FSDP fp32	FSDP fp16
Build/wrap peak	12.82 GiB	12.96 GiB	12.91 GiB
Warmup peak	52.12 GiB	33.18 GiB	28.10 GiB
Measured fwd+bwd peak	52.12 GiB	33.18 GiB	28.09 GiB
Measured step peak	52.12 GiB	33.18 GiB	28.09 GiB
Saving vs DDP (peak)	—	18.93 GiB (36.3%)	24.02 GiB (46.1%)

Timing (median, ms)	DDP	FSDP fp32	FSDP fp16
Total step	1462	1542	507
Forward + backward	1383	1505	455
Gradient sync	18	6	6
Optimizer step	61	31	45
Overhead vs DDP	—	+5.5%	−65.3%

FSDP fp32 adds 5.5% overhead vs DDP despite all-gathering every layer's weights twice per step (forward + backward). With mixed-precision (fp16), FSDP is 65.3% *faster* than DDP fp32 due to halved arithmetic and communication volume. The current-memory table confirms the intended ZeRO-3 behavior: after wrapping, each FSDP rank holds only 6.41 GiB of resident parameters (50% of the full model), after `zero_grad` the fp32 footprint is 19.14 GiB ($3P/N$, because gradients are freed), and after backward it is 25.53 GiB ($4P/N$, with gradients resident). The remaining gap in peak memory comes from transient computation-time allocations rather than persistent replicated model state.

Prefetch-depth sweep. To quantify the memory/latency tradeoff from asynchronous all-gather prefetching, I ran a second experiment with FSDP fp32 and varied the number of future layers prefetched at each layer boundary. The model, batch size, hardware, and benchmark protocol are the same as above.

FSDP fp32 prefetch depth	0	1	2	4
Current after `zero_grad`	19.14 GiB	19.14 GiB	19.14 GiB	19.14 GiB
Measured step peak	32.99 GiB	33.18 GiB	33.36 GiB	33.61 GiB
Peak saving vs DDP	36.7%	36.3%	36.0%	35.5%
Total step latency	1618 ms	1546 ms	1541 ms	1524 ms
Overhead vs DDP	10.6%	5.7%	5.3%	4.2%

The sweep shows the expected tradeoff, now without changing the persistent state size. Since idle full-weight buffers release their storage, all depths have the same 19.14 GiB current footprint after `zero_grad`. Depth only affects transient overlap buffers: increasing it reduces total step time from 1618 ms to 1524 ms, while the measured peak rises modestly from 32.99 GiB to 33.61 GiB. In this setting, `prefetch_depth=1` is a reasonable balanced default: it captures most of the latency gain with only 0.19 GiB additional peak memory, while deeper prefetching gives smaller incremental latency gains and slightly higher peaks.

(b) All-gather timing analysis. For the timing analysis I switched the profiling script to launch `nsys profile` in a subprocess and to bracket exactly one measured training step with `cudaProfilerStart/Stop`. The FSDP implementation also emits NVTX ranges for each prefetch and records the latency of every `work.wait()` call. This is more useful than only looking at aggregate NCCL time: aggregate NCCL kernels tell us communication happened, but `work.wait()` latency tells us whether compute actually stalled waiting for a full weight.

Wait-profile metric	**Value**
Forward all-gather waits	226 / 226 prefetched
Backward all-gather waits	226 / 226 prefetched
Total prefetched wait time	8.822 ms
Average prefetched wait	0.020 ms
Maximum wait	0.043 ms
Synchronous fallback wait	0.000 ms

Figure 4: Nsight System Screenshots.
Nsight timeline plus explicit work.wait() timing show that all-gather is effectively hidden by compute. After learning runtime forward order and adding boundary prefetch, all forward/backward gathers are prefetched and synchronous fallback is eliminated.

The first Nsight trace exposed a bug in my prefetch schedule: every transformer block's `SwiGLU` executes its projections in order `w1 -> w3 -> w2`, while the static module registration order is `w1 -> w2 -> w3`. Because each block has seven sharded linear layers, this mismatch appeared as a regular sync fallback roughly every seven layers. The fix was to learn the runtime forward order during the first warmup pass and then prefetch according to that observed order. I also added boundary prefetching: before the measured forward starts we prefetch the first forward layer, and after forward returns we prefetch the first backward layer. This makes the schedule effectively ring-like, so neither boundary needs a blocking first gather in the measured step.

After these changes, the Nsight run shows that all forward and backward all-gathers are prefetched, including forward layer 0 and the first backward layer. The remaining wait time is only tens of microseconds per layer, and `sync_fallback_total=0.000 ms`. Therefore, on 2×A100-80GB with NVLink, the all-gather communication is effectively hidden behind compute; the FSDP overhead that remains in the end-to-end benchmark is not due to large uncovered all-gather stalls.

7 Analyzing Parallelism Strategies

7.1 Communication Primitives

Problem (alternate_ring_all_reduce)

Here S denotes the communicated tensor size in *bytes*, and W denotes each device's outgoing network bandwidth in *bytes/second* (not a model weight matrix).

$$T = \frac{(N-1)S}{W}$$

The algorithm runs for $N-1$ steps, and in each step every device sends one full tensor of size S bytes over egress bandwidth W, so the total communication time is $T = \frac{(N-1)S}{W}$.

7.2 Data Parallel Calculations

Problem (data_parallel_calcs)

In this subsection, W_1, W_2, W_3 are FFN weight matrices, while plain W without a subscript is the per-device egress bandwidth from Section 8.1. I use S only for a communication payload size in bytes.

(a) FLOPs for backward pass with N_{DP} data parallelism. Under data parallelism, each device receives only a shard of the batch, so the local input and local upstream gradient have shape

$$x^{(i)}, dy^{(i)} \in \mathbb{R}^{B/N_{\mathrm{DP}} \times D}.$$

The weights are not sharded:

$$W_1, W_2 \in \mathbb{R}^{D \times D_{\mathrm{FF}}}, \qquad W_3 \in \mathbb{R}^{D_{\mathrm{FF}} \times D}.$$

We ignore all elementwise operations, so the only FLOPs come from the six matrix multiplications in the backward pass. Using

$$(A, B)(B, C) \to (A, C) \quad \text{costs} \quad 2ABC \text{ FLOPs},$$

the per-device costs are:

$$dz^{(i)} = dy^{(i)} W_3^\top, \qquad \left(\frac{B}{N_{\text{DP}}}, D\right)(D, D_{\text{FF}}) \to \left(\frac{B}{N_{\text{DP}}}, D_{\text{FF}}\right), \qquad \text{FLOPs} = \frac{2BDD_{\text{FF}}}{N_{\text{DP}}}.$$
$$dx_1^{(i)} W_1^\top \text{ part of } dx^{(i)}, \qquad \left(\frac{B}{N_{\text{DP}}}, D_{\text{FF}}\right)(D_{\text{FF}}, D) \to \left(\frac{B}{N_{\text{DP}}}, D\right), \qquad \text{FLOPs} = \frac{2BDD_{\text{FF}}}{N_{\text{DP}}}.$$
$$dx_2^{(i)} W_2^\top \text{ part of } dx^{(i)}, \qquad \left(\frac{B}{N_{\text{DP}}}, D_{\text{FF}}\right)(D_{\text{FF}}, D) \to \left(\frac{B}{N_{\text{DP}}}, D\right), \qquad \text{FLOPs} = \frac{2BDD_{\text{FF}}}{N_{\text{DP}}}.$$
$$dW_3^{(i)} = (z^{(i)})^\top dy^{(i)}, \qquad \left(D_{\text{FF}}, \frac{B}{N_{\text{DP}}}\right)\left(\frac{B}{N_{\text{DP}}}, D\right) \to (D_{\text{FF}}, D), \qquad \text{FLOPs} = \frac{2BDD_{\text{FF}}}{N_{\text{DP}}}.$$
$$dW_2^{(i)} = (x^{(i)})^\top dx_2^{(i)}, \qquad \left(D, \frac{B}{N_{\text{DP}}}\right)\left(\frac{B}{N_{\text{DP}}}, D_{\text{FF}}\right) \to (D, D_{\text{FF}}), \qquad \text{FLOPs} = \frac{2BDD_{\text{FF}}}{N_{\text{DP}}}.$$
$$dW_1^{(i)} = (x^{(i)})^\top dx_1^{(i)}, \qquad \left(D, \frac{B}{N_{\text{DP}}}\right)\left(\frac{B}{N_{\text{DP}}}, D_{\text{FF}}\right) \to (D, D_{\text{FF}}), \qquad \text{FLOPs} = \frac{2BDD_{\text{FF}}}{N_{\text{DP}}}.$$

Therefore the backward-pass compute per device is

$$\boxed{\text{FLOPs}_{\text{DP,bwd,per device}} = \frac{12BDD_{\text{FF}}}{N_{\text{DP}}}}.$$

Equivalently, the aggregate compute across all devices is $12BDD_{\text{FF}}$ FLOPs, but each device only performs the work for its B/N_{DP} local examples.

(b) Communication time for backward pass. In data parallelism, each device computes only a local partial weight gradient from its own B/N_{DP} examples:

$$dW_1^{(i)}, dW_2^{(i)}, dW_3^{(i)}.$$

To obtain the true full-batch gradients, we need to all-reduce these three partial sums across the N_{DP} devices. The activation gradients $dx^{(i)}$ are local to each batch shard and do not need communication.

Each FFN weight-gradient matrix has DD_{FF} elements:

$$dW_1, dW_2 \in \mathbb{R}^{D \times D_{\text{FF}}}, \qquad dW_3 \in \mathbb{R}^{D_{\text{FF}} \times D}.$$

Since the problem assumes FP16 values, each element is 2 bytes. Therefore the total communication payload size, denoted by S, is

$$S = 3 \cdot DD_{\text{FF}} \cdot 2 = 6DD_{\text{FF}} \quad \text{bytes}.$$

Using the ring all-reduce cost from Section 8.1, all-reducing a payload of S bytes across N devices with per-device egress bandwidth W takes

$$2\frac{N-1}{N}\frac{S}{W}$$

seconds. Substituting $N = N_{\text{DP}}$ and $S = 6DD_{\text{FF}}$ gives

$$\boxed{T_{\text{DP,bwd,comm}} = 2\frac{N_{\text{DP}}-1}{N_{\text{DP}}}\frac{6DD_{\text{FF}}}{W} = \frac{12DD_{\text{FF}}}{W}\frac{N_{\text{DP}}-1}{N_{\text{DP}}}}.$$

This communication time is independent of B because data parallelism communicates weight gradients, whose sizes are determined by the model dimensions rather than the local batch size.

(c) Maximum $N_{\mathbf{DP}}$ before communication bottleneck. The backward pass remains compute-bound as long as the communication time is no larger than the local backward compute time:

$$T_{\text{DP,bwd,comm}} \le T_{\text{DP,bwd,compute}}.$$

From part (a), each device performs

$$\frac{12BDD_{\text{FF}}}{N_{\text{DP}}}$$

FLOPs in the backward pass, so with accelerator throughput C FLOP/s,

$$T_{\text{DP,bwd,compute}} = \frac{12BDD_{\text{FF}}}{N_{\text{DP}}C}.$$

From part (b),

$$T_{\text{DP,bwd,comm}} = \frac{12DD_{\text{FF}}}{W}\frac{N_{\text{DP}}-1}{N_{\text{DP}}}.$$

Requiring communication to be hidden by computation gives

$$\frac{12DD_{\text{FF}}}{W}\frac{N_{\text{DP}}-1}{N_{\text{DP}}} \le \frac{12BDD_{\text{FF}}}{N_{\text{DP}}C}.$$

Cancel the common factor $12DD_{\text{FF}}/N_{\text{DP}}$:

$$\frac{N_{\text{DP}}-1}{W} \le \frac{B}{C}.$$

Therefore

$$\boxed{N_{\text{DP}} \le 1 + \frac{BW}{C}}.$$

This bound does not depend on D or D_{FF} because both computation and gradient communication scale linearly with the FFN parameter size DD_{FF}; increasing the batch size B is what increases compute relative to communication and allows scaling to more data-parallel devices.

7.3 FSDP Calculations

Problem (fsdp_calcs)

(a) FLOPs for forward and backward pass with FSDP.

(b) Communication time for forward and backward pass.

(c) Maximum $N_{\mathbf{FSDP}}$ before communication bottleneck.

7.4 Tensor Parallel Calculations

Problem (tp_calcs)

(a) Backward pass equations for TP.

(b) FLOPs for forward and backward pass with TP.

(c) Communication time for forward and backward pass.

(d) Maximum $N_{\mathbf{TP}}$ before communication bottleneck.

7.5 2D Parallelism (FSDP + TP)

Problem (fsdp_tp_calcs)

(a) FLOPs for forward pass with 2D parallelism.

(b) Communication time for forward pass.

(c) Maximum N with overlapped communication.

(d) Maximum N without overlapped communication.

8 Leaderboard

Problem (leaderboard)